

Morphological Skeletonization and Topological Graph Extraction: Algorithmic Paradigms and Pure Function Implementations

Introduction to Spatial Network Extraction in Remote Sensing

The extraction of topological structures from dense raster representations remains a foundational computational challenge in geometric analysis and computer vision. Within the highly specialized context of remote sensing and infrastructure analysis—specifically, the algorithmic processing pipeline designated as `spacenet3_roads_1st/skeletonization`—the primary objective is to systematically translate continuous, probabilistically generated binary masks of road networks into precisely one-pixel-wide, topologically preserved connected lines. This geometric reduction process, formally referred to as morphological skeletonization or medial axis thinning, meticulously strips away volumetric redundancy while retaining the fundamental structural integrity and connectivity of the input shape.¹

Once a binary semantic segmentation mask is successfully reduced to a topological skeleton, a secondary, equally critical challenge emerges: transforming this static pixel-grid representation into an traversable mathematical graph comprising nodes (vertices) and edges. This essential graph conversion facilitates downstream algorithmic traversal, shortest-path routing, topological correction, and complex network analysis.³ The transition from a semantic segmentation output to a fully traversable spatial graph dictates a stringent, two-stage architectural pipeline. The first stage acts as a spatial signal filter applying morphological thinning, while the subsequent stage acts as a spatial parsing traversal to extract the final topological graph.³

For seamless integration into a robust, functional programming-oriented repository such as `sciona-atoms` or `sciona-atoms-dl`, these algorithmic implementations must adhere to the strictest functional programming paradigms. The deployed functions must be mathematically pure, demonstrating absolute referential transparency, consistently deterministic outputs, and complete operational isolation from global state variations, file system input/output interactions, and hidden computational side effects.⁵ This exhaustive analysis dissects the mathematical and topological algorithms behind skeletonization and spatial graph extraction. Furthermore, it rigorously evaluates the `scikit-image` implementation ecosystem for functional purity, resolves the subtle determinism limitations of alternative transform methods, and formally defines the software contracts required to wrap these algorithms as highly isolated,

stateless computational atoms.

Theoretical Foundations of Morphological Thinning

Morphological thinning is predicated on the concept of iterative pixel erosion, driven by highly localized neighborhood evaluations. The paramount mathematical constraint governing any viable thinning algorithm is absolute topological preservation. During the erosion process, the algorithm must never sever a contiguous geometric object into disjointed components, nor may it completely erode a linear object to non-existence, nor create artificial cavities or topological holes that did not exist within the original input mask.⁷

The Zhang-Suen Fast Parallel Thinning Algorithm (1984)

The most prominent, historically significant, and widely adopted algorithm for two-dimensional morphological skeletonization is the Zhang-Suen thinning algorithm, originally introduced to the computer science community in 1984.⁹ The Zhang-Suen approach is an elegant iterative, two-pass (or two-subiteration) parallel thinning algorithm. It operates recursively on a local 3x3 pixel neighborhood, carefully evaluating the eight immediate surrounding pixels of a central target pixel to decisively determine whether that target pixel can be safely deleted—meaning its value is flipped from foreground (1) to background (0)—without disrupting the overarching structure.¹¹

To comprehend the algorithm, one must define the local coordinate topology. Let the central target pixel be denoted as the origin variable P_1 . Its eight surrounding neighbors are arranged in a strict clockwise sequence, beginning with the pixel immediately above it. This establishes the sequence: P_2 (North), P_3 (North-East), P_4 (East), P_5 (South-East), P_6 (South), P_7 (South-West), P_8 (West), and finally P_9 (North-West).¹² Based on this localized topography, the algorithm dynamically calculates two primary metrics for every foreground pixel P_1 . The first metric, $B(P_1)$, represents the total number of non-zero foreground pixels within the eight-pixel neighborhood, calculated mathematically as the sum of P_2 through P_9 .¹² The second metric, $A(P_1)$, serves as a connectivity proxy. It calculates the exact number of zero-to-one (background-to-foreground) transitions encountered when traversing the ordered circular sequence $P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_2$.¹²

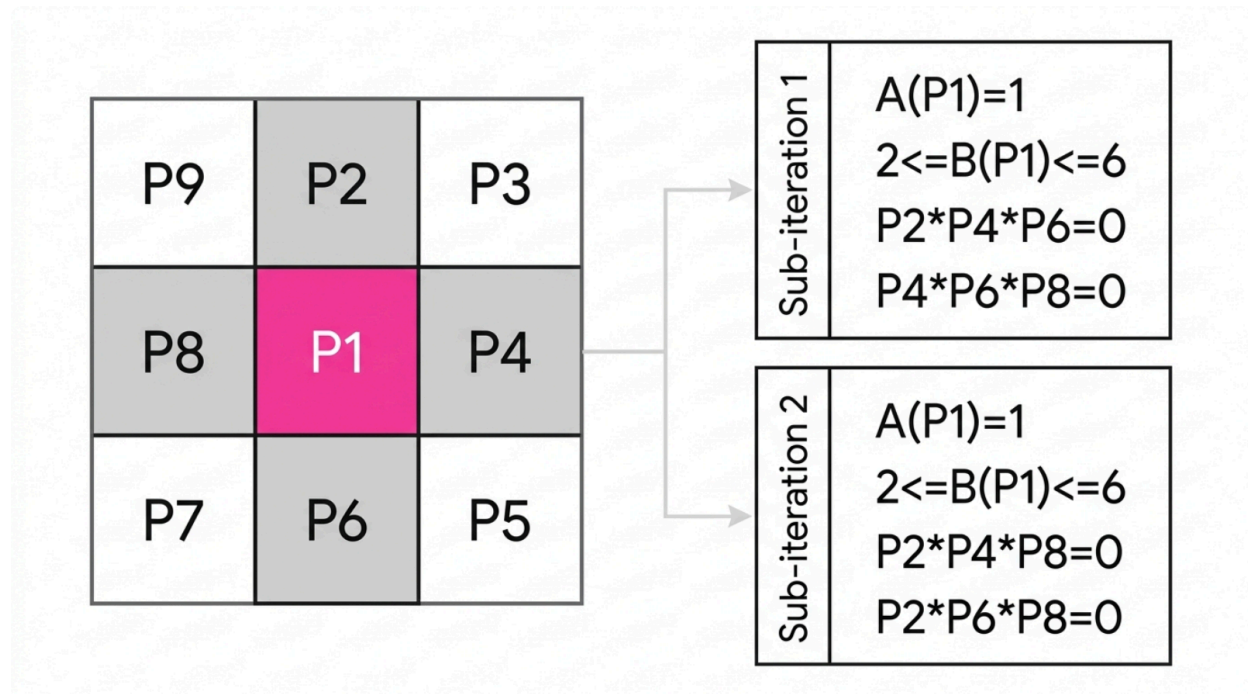
The algorithm systematically flags boundary pixels for deletion over two distinctly alternating sub-iterations, evaluating specific mathematical conditions during each pass to ensure balanced geometric erosion.

Sub-iteration Phase	Mathematical Deletion Constraints	Geometric Purpose
Pass 1	$P_1 =$ $2 \leq$ $A(P_1) =$ $(P_2 \times P_4 \times$ $(P_4 \times P_6 \times$	The algorithm identifies pixels that are not isolated points nor solid internal blocks. The $A(P_1) =$ constraint ensures the pixel is not a critical connecting bridge. The multiplication checks specifically target and erode the South-East boundary points and North-West corner points of the shape.¹⁰
Pass 2	$P_1 =$ $2 \leq$ $A(P_1) =$ $(P_2 \times P_4 \times$ $(P_2 \times P_6 \times$	The connectivity and density constraints remain identical to the first pass. However, the directional multiplication checks are inverted. This phase specifically targets and erodes the North-West boundary points and the South-East corner points.¹⁰

All pixels satisfying the constraints during the evaluation phase of the first sub-iteration are merely flagged in memory; their actual deletion from the raster array must be delayed until the entire image is comprehensively scanned. This delayed mutation is critical to prevent sequential in-place alterations from corrupting the neighborhood logic of adjacent pixels currently being evaluated within the same pass.¹³ Once the first pass concludes and marked pixels are cleared, the second pass begins. An iteration encompassing these two sequential sub-passes is repeated endlessly until the algorithm achieves total convergence—a mathematical state where a full iteration yields zero flagged pixel deletions.¹²

By strategically alternating the directional checks (South-East/North-West boundaries followed by North-East/South-West boundaries), the Zhang-Suen methodology mathematically guarantees that two-pixel-wide linear structures are not entirely eradicated in a single sweeping pass. Instead, it systematically and symmetrically erodes the geometric shape down to a robust, single-pixel medial axis.¹⁰

Zhang-Suen Neighborhood Matrix and Iteration Constraints



The Zhang-Suen algorithm evaluates the 8-neighborhood of each target pixel (P1), utilizing alternating sub-iterations to symmetrically erode object boundaries without severing structural connectivity.

The Lee Thinning Algorithm (1994)

While the Zhang-Suen algorithm is fundamentally optimized for two-dimensional pixel grids, manipulating three-dimensional volumetric spatial data demands a significantly more complex neighborhood evaluation protocol. In 3D space, a 26-neighborhood voxel environment

presents 67,108,864 possible binary configurations (2^{26}), effectively rendering simplistic rule-based boolean checks computationally unfeasible and topologically fragile.⁸

The Lee Thinning Algorithm, formally proposed by T.C. Lee, R.L. Kashyap, and C.-N. Chu in 1994, decisively resolves this multi-dimensional complexity.¹⁶ Rather than relying on rigid kernel-based spatial filters, the Lee algorithm utilizes an advanced octree data structure coupled with a highly optimized decision tree to systematically examine a 3x3x3 volumetric neighborhood.⁵ The core mathematical mechanism governing this algorithm relies on identifying what topology defines as "simple points." A simple point is a surface voxel whose localized deletion does not mathematically alter the Euler characteristic—defined as the number of independent objects minus the number of topological holes—of the local structure.⁷

The Lee algorithm iteratively sweeps the entire volumetric image matrix, applying the decision tree to seamlessly assemble a comprehensive list of candidate removal points. To unequivocally guarantee the preservation of connectivity—given that deleting multiple adjacent "simple points" simultaneously could inadvertently sever a fragile structure—the voxels from the initial candidate list are sequentially rechecked against the evolving neighborhood topology before final deletion is authorized.⁵ Although explicitly engineered to navigate complex 3D manifolds, the algorithm gracefully scales down to operate on 2D inputs, frequently serving as an alternative to Zhang-Suen due to its robust connectivity preservation models and unique final output geometries.⁵

Comparative algorithmic analyses repeatedly indicate that while Zhang-Suen typically demonstrates vastly superior time-complexity execution in strictly 2D environments—largely due to its highly parallelizable nature and reliance on simple, pre-compiled boolean look-up tables (LUT)—variations based on topological decision trees like the Lee method provide excellent structural fidelity in specific use cases.²⁰ However, for the extraction of continuous, two-dimensional road networks derived from remote sensing satellite imagery, the Zhang-Suen methodology remains the unquestioned industry standard due to its unmatched execution speed and superior centerline preservation characteristics for 2D linear structures.¹⁶

The Medial Axis Transform: A Mathematical Alternative

A notable theoretical alternative to iterative morphological thinning is the direct mathematical computation of the Medial Axis Transform (MAT). Originally proposed by researcher Harry Blum in 1964 via his renowned "grassfire analogy," the medial axis is geometrically defined as the specific set of points within a shape that possess at least two distinct closest points located on the object's boundary.²² Under Blum's intuitive analogy, if a uniform fire is simultaneously ignited along the entire outer perimeter of a given geometric shape, the medial axis directly represents the exact loci where the advancing internal fire fronts ultimately collide and quench each other.²³

Within the realm of discrete digital image processing, the MAT is computationally achieved through the rigorous application of a Distance Transform. This transform systematically assigns every foreground pixel a numerical float value representing its exact Euclidean, City-Block, or Chebyshev distance to the nearest existing background pixel.⁵ The medial axis is subsequently isolated and extracted by identifying the local topological maxima—the internal ridges—along this newly generated continuous distance topography.²⁵

Feature Comparison	Iterative Thinning (Zhang-Suen)	Medial Axis Transform (MAT)
--------------------	---------------------------------	-----------------------------

Algorithmic Methodology	Sequential morphological erosion of boundary pixels using neighborhood logic gates. ¹¹	Mathematical calculation of internal distance ridges via Euclidean distance transforms. ⁵
Output Data Value	Yields a strictly binary topological skeleton representing the core structure. ¹⁶	Yields a boolean skeleton array accompanied by a highly valuable floating-point array denoting the exact distance to the boundary at every coordinate. ²⁷
Noise Resilience	Inherently robust against minor boundary fluctuations, maintaining clean, continuous centerlines. ⁵	Extremely sensitive to boundary perturbations. Minor rasterization noise dictates the generation of erratic, long topological branches known as spurs. ²³
Dimensional Flexibility	Routines like Lee94 handle multi-dimensional N-D array arrays natively. ⁵	The optimized scikit-image implementation is currently rigidly constrained to 2D manifolds, failing on volumetric inputs. ³⁰

The profound operational advantage of the MAT over iterative morphological thinning is the retention of localized volumetric data. When computing the medial axis, the algorithm outputs not merely the boolean skeleton, but crucially, an array containing the exact distance transform values correlated to those skeletal coordinates.⁵ For analyzing vast remote sensing road networks, this translates directly to the extraction of the road centerline accompanied by an immediate, mathematically sound estimation of the road's physical width at every individual coordinate along its path.²⁷

However, the MAT suffers from significant mathematical and practical limitations in applied remote sensing. Medial axes are highly sensitive to microscopic boundary perturbations; minor rasterization pixel noise along the edge of an extracted road shape will inevitably spawn long, spurious topological branches (spurs) extending outward to the irregularity, heavily distorting the resulting skeletal graph and confusing downstream routing algorithms.²³ Furthermore, highly optimized standard implementations of the discrete MAT, most notably the implementation contained within scikit-image, remain strictly confined to 2D manifolds, raising execution exceptions if inadvertently confronted with 3D volumetric arrays.³⁰

Implementation Ecosystem: Analyzing scikit-image for Functional Purity

For robust, scalable integration into the sciona-atoms repository, the selected skeletonization algorithms must be carefully sourced from a stable, well-maintained, computationally optimized, and permissively licensed external ecosystem. The scikit-image (skimage.morphology) library swiftly emerges as the optimal target repository. The library is distributed globally under the highly permissive BSD-3-Clause license, rendering its algorithmic contents entirely compatible with enterprise environments, open-source integration, and proprietary commercial usage.³²

The primary architectural research question dictates a rigorous evaluation to determine whether the core scikit-image morphology functions operate as mathematically pure functions, utterly devoid of mutable side effects, hidden global state dependencies, and non-deterministic run-time behavior.

Formal Evaluation of `skimage.morphology.skeletonize`

The skeletonize function exhibits the following high-level operational signature:

`skeletonize(image, *, method=None)`¹⁶

Internally, the skeletonize routine accepts a generic, multi-dimensional numpy ndarray composed of either integers or booleans. It immediately coerces the input into a strict binarized format, actively treating all zero values as the background space and all non-zero values uniformly as the foreground structure.¹⁶

The operational purity of the algorithm is proven through three distinct architectural metrics: First, regarding immutability, the algorithm deliberately allocates a completely novel output array (out) in system memory, populating this new buffer with the resultant thinned skeleton.¹⁶ It absolutely does not mutate the input image array in place unless explicitly and intentionally bypassed via dangerous low-level Cython memory pointers, a mechanism which the high-level Python application programming interface (API) safely shields from the user. Second, regarding determinism, the `method='zhang'` invocation operates through highly deterministic spatial convolutions and fixed boolean Look-Up Tables (LUTs). An array representing all possible 8-neighborhood configurations maps exactly to a static, precompiled 256-element algorithmic array (e.g., `lut = [0, 0, 0, 1, 0, ...]`) that dictatorially controls pixel removal.³⁶ Absolutely no random number generators, stochastic processes, or time-dependent variables are utilized. Third, regarding isolated side effects, the function executes pure CPU-bound mathematical array transformations exclusively. It does not engage in any file system I/O operations, network request calls, database interactions, or mutate persistent global module variables.⁵

Consequently, the `skimage.morphology.skeletonize` function stands as a strictly pure mathematical function. It is inherently safe and structurally ideal to wrap as an isolated signal filter atom within the sciona-atoms pipeline. Reimplementing this complex algorithm from

scratch in native Python is highly discouraged. The scikit-image implementation drastically accelerates the intensive iterative pixel sweeps by utilizing highly optimized, ahead-of-time compiled Cython extensions (specifically `_skeletonize_cy.pyx` and `_skeletonize_various_cy.pyx`) that interface directly and efficiently with contiguous C-level memory buffers.³² A pure native Python and Numpy reimplementation lacking these compiled extensions would suffer catastrophic, exponential performance degradation when tasked with processing the dense, multi-megapixel road masks frequently encountered in Spacenet satellite imagery.¹⁵

Formal Evaluation of `skimage.morphology.medial_axis`

The secondary `medial_axis` function signature presents as: `medial_axis(image, mask=None, return_distance=False, *, rng=None)`²⁵

While `medial_axis` theoretically shares the functional immutability properties of `skeletonize` by faithfully returning completely isolated output arrays (out and the optional `dist`), its deterministic properties present a critical architectural vulnerability that must be addressed.

When processing the internal distance transform, it is mathematically common for multiple distinct pixels to share identical distance and corneriness calculation values. Sorting these massive arrays inevitably creates sorting ties. To handle this, since library version 0.19, scikit-image explicitly introduced a pseudo-random number generator parameter (`rng`) specifically to manage this tie-breaking scenario.³⁹ If the default argument `rng=None` is passed, the function relies on the underlying system entropy to instantly instantiate a new `numpy.random.Generator` (using the PCG64 algorithm) instance.³⁹

If internal tie-breaking occurs during the evaluation of a specific complex road mask, the utter lack of a deterministic random seed will cause the function to unpredictably output microscopically different medial axes across multiple runs on the exact same input geometry. This erratic behavior breaks referential transparency and invalidates the core tenets required of a pure function. To safely wrap `medial_axis` as a deterministic, reproducible atom, the overarching software contract must forcefully overwrite the default stochastic behavior by explicitly passing a constant integer seed (e.g., `rng_seed=42`) into the function call.³⁹

However, due to its well-documented propensity for generating extreme topological spurs in the presence of noise, and the requisite boilerplate handling of pseudorandom seeds, `skeletonize` is strictly favored over `medial_axis` as the primary operational atom for road mask centerline extraction.⁵

Topological Translation: Skeleton to Graph Conversion

Applying morphological skeletonization satisfies only the initial phase of the spatial network extraction pipeline. The direct output of the `skeletonize` atom remains a static 2D grid of pixels. To enable advanced routing algorithms, precise intersection analysis, and high-level structural reasoning, this pixel matrix must be mathematically projected into a formal spatial graph

structure denoted as $G = (V, E)$, where V represents the coordinate vertices (nodes) and

E represents the topological pathways linking them (edges).⁴¹

The Topology of Pixel Neighborhoods

The algorithmic detection of nodes relies intrinsically on computing the connectivity degree of every single foreground pixel residing in the thinned image matrix.⁴ Because the input skeleton has been successfully reduced to a strict one-pixel width by the preceding atom, the degree of any given pixel is easily determined by simply summing the foreground pixels occupying its 8-neighborhood (the 3x3 surrounding matrix window).⁴

Pixel Degree	Topological Classification	Network Interpretation
Degree == 1	Endpoint Node	Pixels possessing exactly one connected neighbor. These represent dead ends, cul-de-sacs, or the terminal points of the road network. ⁴²
Degree == 2	Path / Segment Pixel	Pixels possessing exactly two connected neighbors. These constitute the continuous, unbroken segments of the road. They are mathematically collapsed into edge geometries rather than being treated as distinct graph nodes. ⁴²
Degree ≥ 3	Junction Node	Pixels possessing three or more connected neighbors. These represent vital structural intersections, T-junctions, Y-junctions, and complex cross-roads. ⁴²

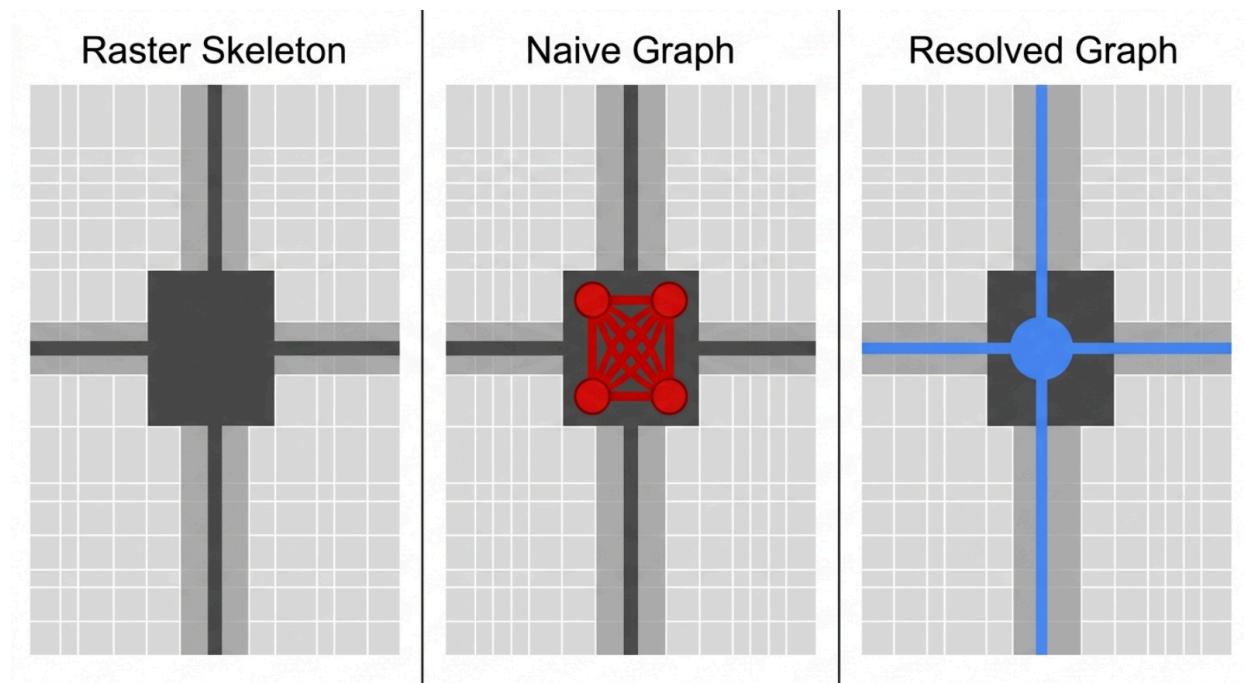
Resolving the Cluster Anomaly

A profound algorithmic complexity in raster-to-graph translation arises specifically at major intersections. In a discrete digital grid, a diagonal cross intersection does not converge perfectly onto a singular isolated pixel. Instead, morphological thinning algorithms operating on

these intersections often yield "junction clusters"—contiguous blocks of 2x2 pixels or highly irregular configurations consisting of multiple pixels all possessing a degree ≥ 3 .⁴⁴

Naive graph conversion algorithms incorrectly interpret each individual high-degree pixel residing within a cluster as a distinct independent node, subsequently connecting them together with sub-pixel micro-edges.⁴⁴ This naive approach introduces severe topological artifacts into the dataset, falsely classifying a standard, physical four-way road intersection as a highly complex, microscopic roundabout composed of multiple tightly intertwined micro-nodes.⁴⁴

Resolving Junction Clusters in Skeleton-to-Graph Conversion



In discrete pixel grids, morphological skeletons often form multi-pixel junction clusters at intersections. Robust graph extraction algorithms collapse these clusters into unified structural nodes via centroid calculation or Minimum Spanning Trees (MST) to preserve accurate network topology.

Ecosystem Evaluation: sknw vs. skan Graph Generation

Two highly capable, open-source Python libraries currently exist to expertly conduct this specific graph projection and successfully resolve the cluster anomaly: sknw (Skeleton Network) and skan (Skeleton Analysis).

The sknw (Skeleton Network) Architecture: The sknw package acts as a lightweight,

BSD-3-Clause licensed library explicitly designed to rapidly parse an N -dimensional skeleton array and deterministically build a standard networkx graph object.³ The primary interface relies on the function `sknw.build_sknw(ske, multi=False)`, which ingests the array and outputs a robust `networkx.Graph` or `networkx.MultiGraph` data structure.³ `sknw` is particularly advantageous because it automatically embeds highly precise numpy coordinate arrays directly within the attributes of the graph object. Network nodes retain a `['pts']` attribute—representing the array of specific pixel coordinates forming the node block—and an `['o']` attribute, representing the calculated floating-point spatial centroid of the junction cluster, effectively resolving the cluster problem.³ Furthermore, the network edges retain a `['pts']` attribute mapping the exact geometric path of the road and a `['weight']` attribute reflecting the calculated float length of the segment.³ The core benefit of `sknw` is its highly focused, mathematically pure execution (yielding deterministic outputs given a constant skeleton matrix) and its direct return of standard networkx primitives immediately traversable by myriad external Python modules.³

The skan (Skeleton Analysis) Architecture: Conversely, `skan` provides a significantly more expansive and complex analytical framework. Rather than jumping straight to a Python-native networkx object, the `skan` architecture constructs an intermediate SciPy Compressed Sparse Row (CSR) matrix via its `skeleton_to_csgraph` function.⁴⁶ To manage the cluster resolution problem and prevent node-bloating at complex intersections, `skan` (specifically since version 0.10) resolves junction clusters by computationally extracting the Minimum Spanning Tree (MST) of the localized junction pixels, subsequently collapsing them mathematically into a unified topological entity.⁴⁴ Under the hood, `skan` is heavily accelerated, utilizing the numba Just-In-Time (JIT) compiler to traverse and process skeletal branches with extreme execution efficiency.⁴⁸ However, the mandatory inclusion of JIT compilation and the reliance on complex pandas DataFrame summarizations (via `skan.summarize`) introduces noticeably heavy dependency footprints into the environment. While the algorithms remain mathematically pure, Numba compilation overhead can introduce initial cold-start latency spikes that are strictly less than ideal for highly concurrent, stateless atomic lambda architectures such as `sciona-atoms`.

Architectural Separation of Concerns

A fundamental architectural question regarding the `sciona-atoms` boundary definitions is whether the morphological skeletonization process and the topological graph extraction process should be fused into a singular, monolithic computational unit.

According to stringent functional software design and robust data architecture principles, these processes must remain absolutely separated into distinct atomic units.⁴⁹ The type signatures fundamentally clash: morphological thinning represents a classic `signal_filter`. Its specific type signature maps a dense spatial array to another spatial array: `(mask: NDArray) -> NDArray`.³ Conversely, graph extraction represents a classic `graph_traversal` operation. Its type signature maps a spatial array to a highly abstract mathematical structure: `(skeleton: NDArray) -> Graph`.³ Combining them violently violates the single-responsibility principle of software

engineering. Furthermore, the intermediate raw pixel-skeleton often provides high utility entirely independent of the abstract graph. It can be strategically utilized for secondary morphological masking, visual overlays, or acting as an input for entirely different distance-transform applications.¹⁶ Encapsulating the graph generation mechanics deeply inside the thinning algorithm destructively hides this valuable intermediate spatial data.

Architectural Design and Atom Specifications for sciona-atoms

Having rigorously established the theoretical parameters and strict functional boundaries of the scikit-image and sknw ecosystems, the following formal specifications formulate the exact software contracts required to safely implement these critical modules within the sciona-atoms repository for the spacenet3_roads_1st/skeletonization stage.

These implementations explicitly target pure functional wrappers. They expressly outlaw in-place memory mutation, demand clear data boundaries, and utilize explicit argument definitions (such as overriding and seeding pseudo-random dependencies) to unconditionally enforce deterministic execution across all environments.

Concept 1: Morphological Thinning Atom (Primary Recommendation)

The Zhang-Suen parallel thinning methodology, cleanly wrapped via the scikit-image API, serves as the optimal primary candidate. It is highly deterministic, incredibly fast across varied hardware, and relies entirely on safe, deterministic C-extensions.¹⁰

YAML

Name: skeletonize_2d

Description: Reduces a dense binary mask to a continuous one-pixel-wide topological skeleton utilizing the Zhang-Suen parallel morphological thinning algorithm. Optimized for 2D road network and centerline extraction.

Source: https://github.com/scikit-image/scikit-image/blob/main/skimage/morphology/_skeletonize.py

License: BSD-3-Clause (scikit-image)

Concept type: signal_filter

Signature: (mask: NDArray) -> NDArray

Pure function boundary: Binary 2D mask in, thinned binary mask out; no file

I/O, global state, or GPU operations. The function executes sequentially within CPU memory limits. It avoids in-place array mutations, allocates a distinct output buffer, involves no stochastic logic, and relies on zero global mutable state.

Contracts:

- require: mask.ndim == 2

- require: mask values are 0 or 1
- ensure: result.shape == mask.shape
- ensure: result is a subset of mask (no new foreground pixels)

Witness: small binary rectangle (10x5 block of ones); verify skeleton is a 1-pixel-wide line along the medial axis.

Dependencies: scikit-image (BSD-3-Clause)

CDG stages covered: spacenet3_roads_1st/skeletonization

Concept 2: Medial Axis Transform Atom (Secondary Alternative)

While structurally powerful due to its unique width measurement capabilities, the Medial Axis Transform is strictly presented as a secondary, alternative structural atom. It is conditionally pure, relying entirely on strict external seed passing by the user to forcefully circumvent the non-deterministic rng tie-breaking mechanisms quietly introduced in scikit-image version 0.19.³⁹

YAML

Name: medial_axis_2d

Description: Computes the medial axis transform of a binary mask by extracting the maximal ridges of the internal distance transform. Evaluates absolute local thickness.

Source: https://github.com/scikit-image/scikit-image/blob/main/skimage/morphology/_skeletonize.py

License: BSD-3-Clause (scikit-image)

Concept type: signal_filter

Signature: (mask: NDArray, return_distance: bool = False, rng_seed: int = 42) -> tuple | NDArray

Pure function boundary: Binary 2D mask in, boolean skeleton and optional distance float array out; no file

I/O, global state, or GPU operations. Pureness is strictly enforced by overriding the default `rng=None` parameter with a hardcoded `rng\_seed` variable to eliminate stochastic tie-breaking during distance ridge sorting.

Contracts:

- require: mask.ndim == 2
- require: mask values are 0 or 1
- require: rng_seed is a constant integer
- ensure: result.shape == mask.shape
- ensure: result is a subset of mask (no new foreground pixels)

Witness: small binary rectangle (10x5 block of ones); verify skeleton is a 1-pixel-wide line along the medial axis.

Dependencies: scikit-image (BSD-3-Clause)

CDG stages covered: spacenet3_roads_1st/skeletonization

Concept 3: Topological Graph Extraction Atom

The structural transition from a static morphological grid into a highly abstract routing representation demands the specialized `skeleton_to_graph` atomic wrapper. For maximum architectural simplicity and minimal dependency overhead, `sknw` is the heavily preferred target framework.³ It minimizes dependencies, directly outputs industry-standard `networkx` objects, and gracefully manages centroid calculations to natively solve the junction cluster anomaly.³

YAML

Name: `skeleton_to_graph`

Description: Translates a morphologically thinned pixel array into an abstract spatial graph network. Evaluates local pixel neighborhoods to detect endpoints (degree == 1) and junctions (degree >= 3), establishing geometric edges from the continuous path pixels (degree == 2). Resolves intersection clusters by averaging spatial centroids.

Source: <https://github.com/Image-Py/sknw/blob/master/sknw/sknw.py>

License: BSD-3-Clause (`sknw`)

Concept type: `graph_traversal`

Signature: (`skeleton: NDArray`) -> `networkx.MultiGraph`

Pure function boundary: Thinned binary 2D mask in, abstract Python object (``networkx.MultiGraph``) out; no file

I/O, global state, or GPU operations. Purely deterministic function relying exclusively on local neighbor counting and distance accumulation algorithms. Maintains no mutable references to the underlying input array memory buffers.

Contracts:

- require: `skeleton.ndim == 2`
- require: skeleton values are 0 or 1
- require: skeleton is previously thinned (maximum path width == 1 pixel)
- ensure: `graph.nodes()` contain continuous spatial 'pts' arrays and 'o' centroids
- ensure: `graph.edges()` contain continuous spatial path arrays and positive float lengths ('weight')

Witness: Pass a binary cross mask ('+' shape) with arm length 5. The output graph must exhibit precisely five nodes: four discrete endpoints (degree 1) mapping to the extremities, and exactly one central node mathematically positioned at the cross intersection. The graph must contain exactly four edges connecting the endpoints to the core.

Dependencies: `sknw` (BSD-3-Clause), `networkx` (BSD-3-Clause)

CDG stages covered: `spacenet3_roads_1st/graph_extraction`

Conclusions and Pipeline Implementation Strategy

The successful, automated translation of satellite-derived road network raster masks into highly actionable, routing-capable spatial graphs necessitates a rigidly separated two-stage computational pipeline.

For the initial `spacenet3_roads_1st/skeletonization` stage, the deployment of the `skimage.morphology.skeletonize` function wrapper is strictly mandated. The underlying Zhang-Suen morphological thinning algorithm demonstrably offers the greatest computational efficiency, memory safety, and precise linear topological preservation available for massive 2D road matrices.¹⁶ Because the Zhang-Suen implementation relies exclusively on highly deterministic Boolean look-up tables and iterative, memory-safe grid sweeps processed via compiled C-extensions, it perfectly conforms to the strict, pure-function contracts demanded by the scalable `sciona-atoms` ecosystem.⁵ Attempting to substitute this specific process with the secondary `medial_axis` function introduces entirely unnecessary risks regarding deterministic execution via its internal random tie-breaking mechanisms³⁹ and its heightened mathematical sensitivity to minor boundary noise that consistently spawns spurious, non-existent road branches.²³

Following the successful reduction of the dense binary mask to a continuous skeletal array, the topological architecture must then be parsed via a distinctly separate `skeleton_to_graph` atom. Utilizing the `sknw` library over heavier alternatives like `skan` ensures an elegant, lightweight transition directly into a usable `networkx` object, efficiently circumventing known junction-cluster issues at physical intersection points by merging contiguous high-degree pixels into highly accurate spatial centroids.³ Keeping these distinct functional processes architecturally separated forcefully safeguards the single-responsibility principle of the repository, enabling optimal functional scaling, highly targeted unit testing using 10x5 block witnesses, and guaranteeing deterministic reproducibility in complex remote sensing pipeline analytics.

Works cited

1. Analysis of Iterative Skeletonization Algorithm in Recognizing Medial Axis of a Binary Image - International Journal of Engineering Research & Technology, accessed May 1, 2026, <https://www.ijert.org/research/analysis-of-iterative-skeletonization-algorithm-in-recognizing-medial-axis-of-a-binary-image-IJERTV3IS111060.pdf>
2. Skeleton-based automatic road network extraction from an orthophoto colored point cloud - ACRS2019, accessed May 1, 2026, <http://acrs2019.sigongji.com/wp/pdf/MoE2-3.pdf>
3. Image-Py/sknw: build net work from skeleton image (2D-3D) - GitHub, accessed May 1, 2026, <https://github.com/Image-Py/sknw>
4. ykaranouskaya/Skeleton_to_graph-labeling-: The second way to create the graph from the 3D skeleton. - GitHub, accessed May 1, 2026, https://github.com/ykaranouskaya/Skeleton_to_graph-labeling-
5. Skeletonize — skimage 0.25.2 documentation, accessed May 1, 2026,

- https://scikit-image.org/docs/0.25.x/auto_examples/edges/plot_skeleton.html
6. Skeletonize — skimage 0.26.0 documentation, accessed May 1, 2026,
https://scikit-image.org/docs/stable/auto_examples/edges/plot_skeleton.html
 7. (PDF) Implementation of a 3D thinning algorithm - ResearchGate, accessed May 1, 2026,
https://www.researchgate.net/publication/28359528_Implementation_of_a_3D_thinning_algorithm
 8. Implementation of a 3D thinning algorithm, accessed May 1, 2026,
<https://www.sci.utah.edu/~dmw/FEMesher/references/ITKbinaryThinningImageFilter3D.pdf>
 9. A fast parallel algorithm for thinning digital patterns - Semantic Scholar, accessed May 1, 2026,
<https://www.semanticscholar.org/paper/A-fast-parallel-algorithm-for-thinning-digital-Zhang-Suen/950c8d0b042b2a55a3f32fa9bba9af485a26ad8a>
 10. A Fast Parallel Algorithm for Thinning Digital Patterns, accessed May 1, 2026,
http://agcggs680.pbworks.com/f/Zhan-Suen_algorithm.pdf
 11. The Zhang Suen Thinning Algorithm, accessed May 1, 2026,
https://yaikhom.com/pdfs/zhang_suen_thinning_algorithm.pdf
 12. Zhang-Suen thinning algorithm - Rosetta Code, accessed May 1, 2026,
https://rosettacode.org/wiki/Zhang-Suen_thinning_algorithm
 13. Skeletonization by Zhang-Suen Thinning Algorithm, Python and Matlab Implementation - GitHub, accessed May 1, 2026,
<https://github.com/linbojin/Skeletonization-by-Zhang-Suen-Thinning-Algorithm>
 14. Fast parallel algorithm for thinning digital patterns (Journal Article) | OSTI.GOV, accessed May 1, 2026, <https://www.osti.gov/biblio/5085064>
 15. Zhang-Suen Thinning Algorithm - AWS, accessed May 1, 2026,
http://rstudio-pubs-static.s3.amazonaws.com/302782_e337cfbc5ad24922bae96ca5977f4da8.html
 16. skimage.morphology — skimage 0.26.0 documentation, accessed May 1, 2026,
<https://scikit-image.org/docs/stable/api/skimage.morphology.html>
 17. 22. Skeletonizing an image — Hands-on introduction to Scikit-image methods, accessed May 1, 2026,
<https://tirthajyoti.github.io/Scikit-image-book/Skeletonize.html>
 18. Skeletonize — skimage v0.13.1 docs - Developer's Documentation Collections, accessed May 1, 2026,
http://devdoc.net/python/scikit-image-doc-0.13.1/auto_examples/edges/plot_skeleton.html
 19. skimage.morphology — skimage 0.25.2 documentation, accessed May 1, 2026,
<https://scikit-image.org/docs/0.25.x/api/skimage.morphology.html>
 20. A Comparative Study of Fingerprint Thinning algorithms. - ResearchGate, accessed May 1, 2026,
https://www.researchgate.net/publication/220803321_A_Comparative_Study_of_Fingerprint_Thinning_algorithms
 21. japgarido/Skeletonization-Methods-comparison - GitHub, accessed May 1, 2026,
<https://github.com/japgarido/Skeletonization-Methods-comparison>

22. Medial Meshes – A Compact and Accurate Representation of Medial Axis Transform - HKU, accessed May 1, 2026,
<https://i.cs.hku.hk/~zyyu/publication/medialmesh-tvcg2016.pdf>
23. Extended Grassfire Transform on Medial Axes of 2D Shapes - WashU Computer Science & Engineering, accessed May 1, 2026,
https://www.cs.wustl.edu/~taoju/research/ma_final.pdf
24. COMPARISON OF THINNING ALGORITHMS FOR VECTORIZATION OF ENGINEERING DRAWINGS - Journal of Theoretical and Applied Information Technology, accessed May 1, 2026,
<http://www.jatit.org/volumes/Vol94No2/2Vol94No2.pdf>
25. Module: morphology — skimage v0.10dev docs, accessed May 1, 2026,
<https://sharky93.github.io/docs/dev/api/skimage.morphology.html>
26. medial_axis - Document, accessed May 1, 2026,
https://pydocs.github.io/p/skimage/0.17.2/api/skimage.morphology._skeletonize.medial_axis.html
27. Medial axis skeletonization — skimage v0.11dev docs, accessed May 1, 2026,
http://sharky93.github.io/docs/gallery/auto_examples/plot_medial_transform.html
28. Thinning algorithm : r/compsci - Reddit, accessed May 1, 2026,
https://www.reddit.com/r/compsci/comments/4lmh20/thinning_algorithm/
29. Morphology - Skeletonization/Medial Axis Transform, accessed May 1, 2026,
<https://homepages.inf.ed.ac.uk/rbf/HIPR2/skeleton.htm>
30. Does medial axis skeletonization work for 3D input arrays ? #5468 - GitHub, accessed May 1, 2026, <https://github.com/scikit-image/scikit-image/issues/5468>
31. plot_skeleton.rst.txt - Scikit-image, accessed May 1, 2026,
https://scikit-image.org/docs/stable/_sources/auto_examples/edges/plot_skeleton.rst.txt
32. License — skimage 0.25.2 documentation, accessed May 1, 2026,
<https://scikit-image.org/docs/0.25.x/license.html>
33. License — skimage 0.26.0 documentation, accessed May 1, 2026,
<https://scikit-image.org/docs/stable/license.html>
34. The 3-Clause BSD License - Open Source Initiative, accessed May 1, 2026,
<https://opensource.org/license/bsd-3-clause>
35. Module: morphology — skimage v0.6dev docs - Tony Yu, accessed May 1, 2026,
<http://tonysyu.github.io/scikit-image/api/skimage.morphology>
36. morphology.skeletonize junction handling · Issue #6828 - GitHub, accessed May 1, 2026, <https://github.com/scikit-image/scikit-image/issues/6828>
37. scikit-image - Oracle Help Center, accessed May 1, 2026,
https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsl/IPython_Development_Team_and_Jupyter_Development_Team_scikit-image.html
38. Skeletonide is a parallel implementation of Zhang-Suen morphological thinning algorithm written in Halide-lang. Use it for fast skeletonization of binary masks on the GPU. · GitHub, accessed May 1, 2026,
<https://github.com/postmalloc/skeletonide>
39. skimage.morphology — skimage 0.26.0 documentation - Scikit-image, accessed May 1, 2026,

https://scikit-image.org/docs/stable/api/skimage.morphology.html#skimage.morphology.medial_axis

40. scikit-image 0.19.3 release notes — skimage 0.21.0 documentation, accessed May 1, 2026, https://scikit-image.org/docs/0.21.x/release_notes/release_0.19.html
41. GraphMorph: Tubular Structure Extraction by Morphing Predicted Graphs - arXiv, accessed May 1, 2026, <https://arxiv.org/pdf/2502.11731>
42. AnalyzeSkeleton - ImageJ Wiki, accessed May 1, 2026, <https://imagej.net/imagej-wiki-static/AnalyzeSkeleton>
43. Calculating a graph from a skeletonized image - Stack Overflow, accessed May 1, 2026, <https://stackoverflow.com/questions/57029293/calculating-a-graph-from-a-skeletonized-image>
44. FAQ — Skan 0.13.1 documentation, accessed May 1, 2026, <https://skeleton-analysis.org/stable/misc/faq.html>
45. Handle package sknw in python - Stack Overflow, accessed May 1, 2026, <https://stackoverflow.com/questions/46601739/handle-package-sknw-in-python>
46. Getting started: Skeleton analysis with Skan — Skan 0.13.1 documentation, accessed May 1, 2026, https://skeleton-analysis.org/stable/getting_started/getting_started.html
47. skan/doc/getting_started/getting_started.md at main · jni/skan - GitHub, accessed May 1, 2026, https://github.com/jni/skan/blob/main/doc/getting_started/getting_started.md
48. Skeleton analysis - Dask Working Notes, accessed May 1, 2026, <https://blog.dask.org/2021/05/07/skeleton-analysis>
49. GraphMorph: Tubular Structure Extraction by Morphing Predicted Graphs - arXiv, accessed May 1, 2026, <https://arxiv.org/html/2502.11731v1>
50. Medial axis skeletonization — skimage v0.6dev docs - Tony Yu, accessed May 1, 2026, http://tonysyu.github.io/scikit-image/auto_examples/plot_medial_transform.html
51. scikit-image 0.19.3 (2022-06-12) — skimage 0.26.0 documentation, accessed May 1, 2026, https://scikit-image.org/docs/stable/release_notes/release_0.19.html